CHAPTER 2 LITERATURE REVIEW

This chapter reviews the principal numerical ingredients involved in aircraft ice-accretion simulations. It first presents the conventional quasi-steady framework adopted by most icing solvers before reviewing the aerodynamic, dispersed-phase, thermodynamic, geometry-evolution, and geometry-representation models and techniques reported in the literature. The chapter concludes with a synthesis of the current state of the art and identifies the remaining challenges that motivate the developments presented in the following chapters.

2.1 Overview of Numerical Ice Accretion Modeling

Aircraft icing is the accumulation of ice on aircraft surfaces caused by the impingement and freezing of supercooled water droplets. From a numerical point of view, the problem involves the coupling of an external aerodynamic flow, a dispersed liquid phase, a surface thermodynamic balance, and an evolving fluid-solid interface. These different components are characterized by distinct physical time scales, which motivates the modular formulation used in most ice-accretion solvers [25]. Certification icing conditions generally involve exposure over several minutes, resulting in an ice-accretion time scale that is several orders of magnitude longer than the characteristic flow and droplet time scales [25]. Owing to this disparity, the transient ice-accretion process may be approximated as a succession of quasi-steady states.

In addition to this separation of time scales, the low concentration of liquid water in icing clouds permits a further simplification in the treatment of the dispersed phase. Certification icing envelopes involve standard supercooled droplets (SSD) with characteristic diameters ranging from approximately 10 to 50 μm, while SLD conditions can extend to diameters approaching 1000 μm. For a typical liquid water content (LWC) of 1 g/m^3, the corresponding liquid volume fraction is of the order of 10^{-6} [33, 34]. This small volume fraction justifies the common assumption of one-way coupling between the air and droplet phases.

These assumptions provide the foundation for the conventional quasi-steady ice-accretion framework (Fig. 2.1). In its simplest form, the aerodynamic flow is first computed around the clean geometry. The resulting velocity field is then used to determine the transport of the dispersed phase and the local droplet impingement rate on the exposed surface. The latter is then provided to a surface thermodynamic model, which evaluates the local mass and energy balances and returns the ice-growth rate. Finally, the geometry is updated

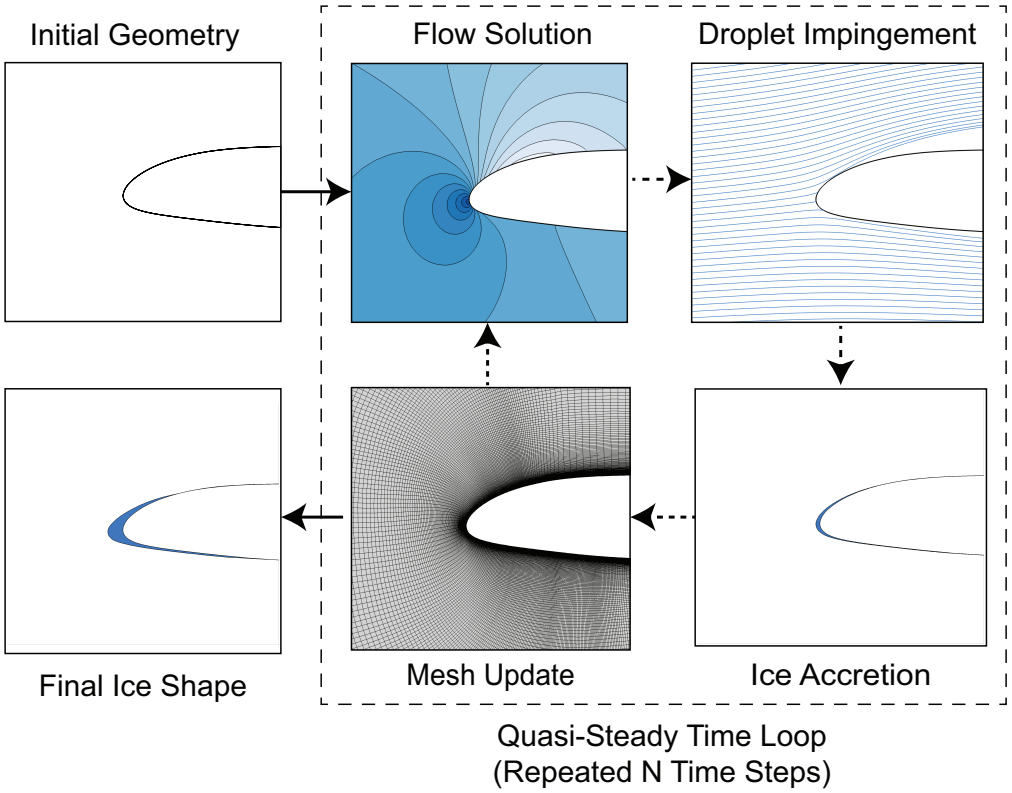


Figure 2.1 Schematic of the multilayer ice accretion framework with representative solutions at each stage.

according to the predicted ice thickness. Representative aerodynamic, droplet, geometry-evolution, and mesh-update solutions associated with this sequence are shown in Fig. 2.1. When this sequence is performed only once over the full exposure time, the approach is often referred to as a single-step or single-shot accretion simulation. When the total icing time is divided into several shorter intervals, the same sequence is repeated on the evolving geometry, leading to a multilayer formulation. Intermediate strategies, such as predictor–corrector formulations [35, 36], have also been proposed to partially account for the retroaction of the growing ice shape on the flow and droplet fields. This limits the additional computational cost to a small number of surface-solution correction cycles.

Although the quasi-steady multilayer approach remains the standard strategy in engineering ice-accretion simulations, the physical process is inherently unsteady. Fully time-accurate formulations have been reported in the literature [37]. More recently, unsteady couplings involving scale-resolving flow simulations, such as LES, have also been explored [38]. These approaches provide a more direct representation of the continuous interaction between the flow and the evolving ice surface, but their computational cost remains prohibitive for routine

industrial icing applications, especially in 3D configurations.

To provide a broader perspective on existing numerical approaches, Table 2.1 summarizes several state-of-the-art ice-accretion solvers reported in the literature [27, 28]. The table highlights the main numerical ingredients used in each framework. The remainder of this chapter then examines these components in more detail.

Table 2.1 Comparison of representative numerical aircraft icing solvers.

Org.	Solver*	Flow	Droplet	Thermo.	Geo.	IBM	Multishot	Ref.
ANSYS Canada	FENSAP-ICE	RANS	Eul.	Messinger	Algebraic + Wrapping	✗	✓	[39]
CIRA	SIMBA	RANS	Eul.	Messinger	Algebraic	✓	✓	[31]
NASA	GlennICE	RANS/Euler	Lag.	Messinger	Algebraic	✗	✓	[40]
ONERA	IGLOO3D	RANS/Euler	Eul.	Messinger	Algebraic/LS	✓	✓	[30, 32, 35]
Politecnico di Milano	PoliMIce	RANS	Lag.	Mod. Myers	Algebraic + LS	✗	✓	[41]
Polytechnique Montréal**	CHAMPS	RANS/Euler/Potential	Eul.	Messinger/SWIM	Algebraic + Hyp.	✗	✓	[2, 5]
Uni. of Strasbourg	NSMB	RANS	Eul.	SWIM	LS	✓	✓	[42]

Abbreviations: Eul.: Eulerian; Lag.: Lagrangian; LS: Level Set; Hyp.: Hyperbolic; Mod.: Modified; IBM: Immersed-Boundary Method.
Notes:
* The solvers listed here refer to the icing frameworks. The aerodynamic flow solution may be provided by separate CFD solvers; therefore, the Flow column indicates the types of flow formulations compatible with each icing framework rather than a specific flow solver.
** Prior to the developments presented in this thesis, multilayer ice-accretion simulations in CHAMPS were limited to 2D/2.5D configurations and exhibited robustness limitations, while 3D applications were restricted to single-step accretion.

2.2 Aerodynamics

Early theoretical developments in aircraft icing were established before the availability of modern computational resources. Foundational contributions by Langmuir and Blodgett, Hardy, and Messinger provided the theoritical basis for ice accretion modeling [43]. However, these early models were restricted to simple configurations, such as cylinders and spheres, or relied on empirical corrections when analytical solutions were not available. The extension of icing analysis to more representative aerodynamic geometries became possible only with the development of computational methods.

The first generation of numerical icing tools emerged from research efforts led mainly by NASA Lewis Research Center in the United States, Polytechnique Montréal in Canada, and ONERA in France [43]. Early codes such as LEWICE [44] and CANICE [45] relied primarily on inviscid aerodynamic descriptions, typically based on potential-flow or panel methods, coupled with boundary-layer corrections to estimate the near-wall quantities required by the icing model. Similarly, ONERA developed icing tools based on Euler flow solutions combined with boundary-layer calculations [46]. These approaches provided an efficient framework for ice-accretion prediction and formed the basis of several widely used engineering tools. The capabilities of these early numerical icing tools were subsequently assessed through international benchmarking efforts, notably the NATO/RTO exercise on “Assessment of Icing Code Prediction Capabilities,” which culminated in a workshop held at CIRA in Capua, Italy, in 2000. By comparing blind numerical predictions against common experimental icing

cases, this initiative provided a coordinated international assessment of existing icing codes and helped identify limitations and directions for their further development [47].

With the maturation of computational fluid dynamics (CFD), especially from the 1970s onward, Reynolds-averaged Navier–Stokes (RANS) solvers progressively became the dominant aerodynamic model for practical icing simulations [48]. RANS solvers provide a more complete description of viscous effects and near-wall flow features, which are important for the ice shape prediction. This transition is reflected in recent IPW, where the majority of reported simulations rely on RANS-based aerodynamic solutions, most often using turbulence closures such as the Spalart–Allmaras or k–ω models [49, 50]. Although RANS calculations are more expensive than inviscid approaches, they remain affordable for industrial applications and provide a practical compromise between physical fidelity and computational cost.

At the same time, recent work has renewed interest in lower-cost inviscid formulations strongly coupled with boundary-layer models [30, 32, 51]. These approaches aim to recover part of the accuracy of RANS-based icing simulations [52, 53] while retaining the computational efficiency of Euler or potential-flow solvers. Such strategies are particularly attractive for design studies, parametric analyses, and multilayer ice-accretion calculations. The current literature therefore reflects two complementary trends: the continued use of RANS solvers as the standard aerodynamic model for engineering icing simulations, and the renewed interest in revisiting earlier inviscid and low- to medium-fidelity approaches using the knowledge and reference data gained from high-fidelity computational tools.

An important output of the aerodynamic solution for ice-accretion calculations is the convective heat-transfer coefficient (HTC). Its accurate evaluation is particularly important because convection generally represents one of the dominant contributions to the surface energy balance. The convective heat flux may be expressed as

$$\dot{Q}_c = HTC\,(T_s - T_{rec}) \tag{2.1}$$

where T_s is the surface temperature and T_{rec} the recovery temperature. The HTC may be estimated from empirical correlations, such as those based on the Reynolds analogy and skin-friction coefficient [54]. Alternatively, it may be obtained directly from the aerodynamic solution by imposing an isothermal wall condition and evaluating the resulting wall heat flux. In this case, the recovery temperature can be approximated as

$$T_{rec} = T_e \left(1 + r\frac{\gamma - 1}{2} M_e^2\right) \tag{2.2}$$

where T_e and M_e are the temperature and Mach number at the edge of the boundary layer, respectively, with M_e obtained from the local flow conditions using isentropic relations [35], and r is the recovery factor. Montreuil et al. [55] use $r = Pr^{1/3} \approx 0.9$ for turbulent flows and $r = 0.85$ for laminar flows.

To avoid relying on an approximate recovery factor, Montreuil et al. [55] also proposed evaluating the HTC from two RANS solutions performed with different thermal wall conditions. For two prescribed wall temperatures $T_{w,0}$ and $T_{w,1}$, the corresponding wall heat fluxes satisfy

$$\dot{Q}_{w,0} = HTC\,(T_{w,0} - T_{rec}) \qquad \dot{Q}_{w,1} = HTC\,(T_{w,1} - T_{rec}) \tag{2.3}$$

Assuming that the HTC and recovery temperature remain unchanged between the two solutions, subtraction eliminates T_{rec}, giving

$$HTC = \frac{\dot{Q}_{w,1} - \dot{Q}_{w,0}}{T_{w,1} - T_{w,0}} \tag{2.4}$$

One of the solutions may alternatively be performed with an adiabatic wall, for which $\dot{Q}_{w,0} = 0$ and the resulting wall temperature directly provides T_{rec}. This two-solution procedure therefore allows both the HTC and recovery temperature to be obtained directly from the CFD solution without relying on empirical recovery-temperature correlations. However, it approximately doubles the aerodynamic computational cost, which limits its practicality in repeated ice-accretion calculations. Consequently, most icing solvers rely on a modeled recovery temperature together with a single thermal flow solution.

2.3 Dispersed Phase Modeling

Droplet-impingement prediction provides the local impingement rate required by the thermodynamic stage. Before the local ice-growth rate can be evaluated, the amount of supercooled water reaching the surface must first be determined. This quantity is commonly expressed through the collection efficiency, denoted by β, a dimensionless variable that represents the ratio of the local impinging water mass flux to the freestream water mass flux. The dimensional impinging water mass flux can therefore be written as

$$\dot{m}_{\text{imp}} = \beta\,\text{LWC}\,U_\infty \tag{2.5}$$

where U_∞ is the freestream velocity and LWC is the Liquid Water Content. The collection

efficiency directly determines the water mass available for freezing or runback and therefore strongly influences the predicted ice shape.

The definition of β depends on the droplet formulation. In an Eulerian framework, the droplet phase is treated as a continuous medium. The local collection efficiency is then obtained from the normal component of the droplet mass flux at the wall [56]. In nondimensional form, it can be expressed as

$$\beta = -\alpha \, \boldsymbol{u}_d \cdot \boldsymbol{n} \tag{2.6}$$

where α is the nondimensional water concentration, $\boldsymbol{u}_d$ is the droplet velocity, and $\boldsymbol{n}$ is the outward unit normal to the surface. In a Lagrangian framework, the collection efficiency is instead determined by tracking the deformation of a droplet streamtube initialized in the freestream and tracked until it intersects the surface. As illustrated in Fig. 2.2, β may be interpreted, for a given surface element, as the ratio between the upstream seeding area associated with a set of neighboring droplet trajectories and the corresponding impingement area on the wall [57]:

$$\beta = \frac{A_{\text{upstream}}}{A_{\text{impingement}}} \tag{2.7}$$

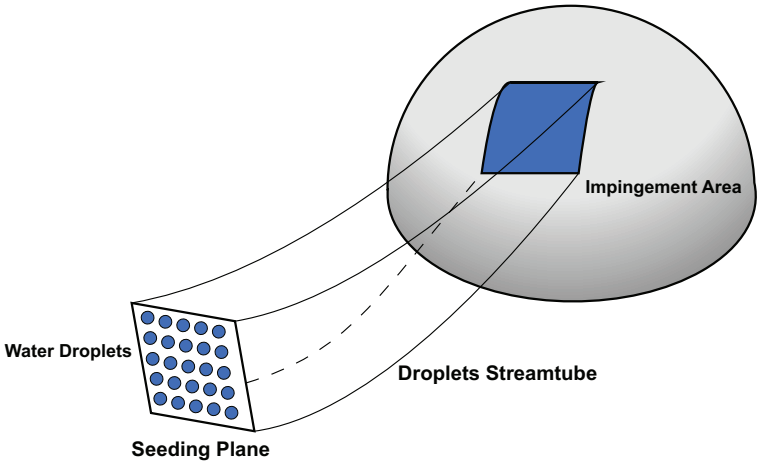


Figure 2.2 Schematic representation of the Lagrangian droplet-seeding procedure. Droplets are released from an upstream plane and tracked until they impinge on the surface.

Historically, Lagrangian droplet tracking was the only approach in early icing codes [44, 45]. In solvers based on potential-flow or panel methods, the airflow field was inexpensive to evaluate and droplet trajectories could be integrated directly through the computed velocity

field. This made the Lagrangian formulation attractive because it provided a clear physical description of droplet motion. However, as ice-accretion solvers evolved toward Euler and RANS aerodynamic models, the computational cost of Lagrangian tracking became more restrictive. Accurate impingement maps require a large number of droplets, particularly in 3D configurations. In addition, each droplet trajectory requires repeated interpolation of the carrier-flow solution, localization of the droplet within the computational mesh, and detection of surface impacts. These operations can become expensive and difficult to parallelize efficiently on complex unstructured grids. These limitations motivated the development of Eulerian droplet formulations [56], in which the dispersed phase is solved as a continuum using partial differential equations (PDE).

2.3.1 Eulerian Droplet Formulation

Eulerian dispersed-phase models describe the droplet field through continuous variables defined over the computational domain [56]. The main unknowns are the nondimensional water concentration α and the droplet velocity $\boldsymbol{u}_d$. A common formulation consists of a conservation equation for the droplet concentration

$$\frac{\partial \alpha}{\partial t} + \nabla \cdot (\alpha \boldsymbol{u}_d) = 0 \tag{2.8}$$

combined with a momentum equation for the droplet phase

$$\frac{\partial (\alpha \boldsymbol{u}_d)}{\partial t} + \nabla \cdot (\alpha \boldsymbol{u}_d \otimes \boldsymbol{u}_d) = \frac{C_D Re_D}{24 S_t} \alpha (\boldsymbol{u}_a - \boldsymbol{u}_d) + \alpha \left(1 - \frac{\rho_a}{\rho_d}\right) \frac{\boldsymbol{g}}{Fr^2} \tag{2.9}$$

where $\boldsymbol{u}_a$ is the airflow velocity, C_D is the droplet drag coefficient, Re_D is the droplet Reynolds number, S_t is the droplet Stokes number, ρ_a and ρ_d are the air and droplet densities, respectively, $\boldsymbol{g}$ is the gravitational acceleration vector, and Fr is the Froude number. The nondimensional parameters are defined as

$$\mathrm{Re}_D = \frac{\rho_a |\boldsymbol{u}_a - \boldsymbol{u}_d| D}{\mu_a}, \tag{2.10}$$

$$S_t = \frac{\tau_d}{\tau_f} = \frac{\rho_d D^2 U_\infty}{18 \mu_a c_{ref}}, \tag{2.11}$$

and

$$\mathrm{Fr} = \frac{U_\infty}{\sqrt{Lg}}, \tag{2.12}$$

where D is the droplet diameter, μ_a is the dynamic viscosity of air, U_∞ is the freestream velocity, c_{ref} is the reference length, $\tau_d = \rho_d D^2/(18\mu_a)$ is the characteristic droplet response time, $\tau_f = c_{ref}/U_\infty$ is the characteristic flow timescale, and $g = |\boldsymbol{g}|$ is the magnitude of the gravitational acceleration.

The Eulerian approach is well suited to modern finite-volume CFD solvers. Since the droplet equations are written in conservative form, they can be discretized using numerical techniques similar to those used for the airflow equations. This makes the approach relatively efficient, naturally compatible with parallel computing, and convenient for 3D simulations on complex grids.

Nevertheless, the Eulerian formulation has limitations. Since the droplet phase is averaged within each computational cell, individual droplet trajectories, impact velocities, and impact angles are not directly available. Moreover, numerical diffusion may smear the impingement limits, especially near shadow zones. Numerical oscillations may also appear in low-concentration regions and can lead to nonphysical values of α. Additional numerical treatments are therefore often required to preserve positivity, reduce artificial diffusion, and improve conservation properties [58, 59].

2.3.2 Lagrangian Droplet Formulation

In a Lagrangian formulation, droplets are tracked individually by integrating their equations of motion through the carrier-flow field. The droplet velocity $\boldsymbol{u}_d$ and position $\boldsymbol{x}_d$ satisfy

$$\frac{d\boldsymbol{u}_d}{dt} = \frac{\boldsymbol{f}}{m} + \boldsymbol{g} = F\left(\boldsymbol{u}_a - \boldsymbol{u}_d\right) + \left(1 - \frac{\rho_a}{\rho_d}\right)\boldsymbol{g} \tag{2.13}$$

and

$$\frac{d\boldsymbol{x}_d}{dt} = \boldsymbol{u}_d \tag{2.14}$$

where $\boldsymbol{f}$ is the aerodynamic force acting on the droplet, and m is the droplet mass. The scalar drag coefficient F is given by

$$F = \frac{3C_D\rho_a\left|\boldsymbol{u}_a - \boldsymbol{u}_d\right|}{4\rho_d D} \tag{2.15}$$

The airflow velocity $\boldsymbol{u}_a$ is evaluated at the instantaneous droplet position.

Although Lagrangian methods are considered expensive for high-fidelity icing simulations, they have recently gained renewed interest [60–64]. Modern high-performance computing resources, improved particle-localization algorithms, and the need for detailed impact information have made Lagrangian approaches increasingly attractive. This is particularly true for SLD conditions, where splashing, bouncing, and the motion of ejected droplets resulting from splashing are more naturally described within the Lagrangian framework.

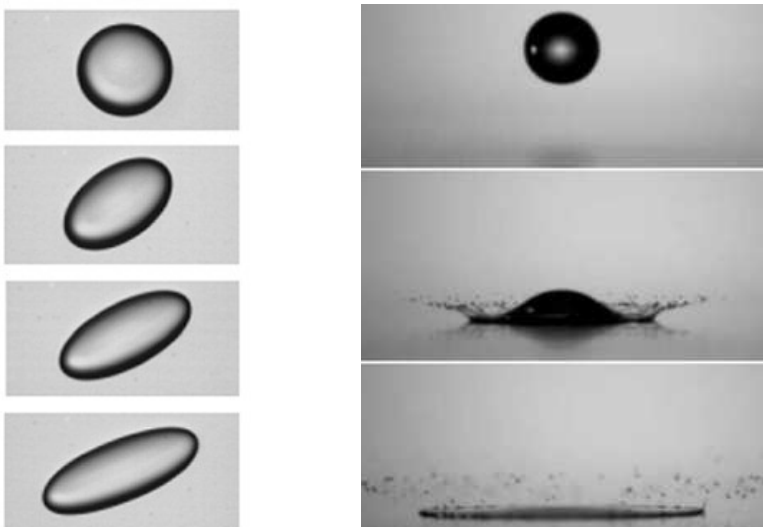
2.3.3 Supercooled Large Droplet Modeling

Supercooled Large Droplet (SLD) conditions are characterized by droplet sizes substantially larger than those typically encountered under Standard Supercooled Droplet (SSD) conditions. As indicated by the Stokes number defined in Eq. (2.11), the response of a droplet to changes in the surrounding airflow depends strongly on its diameter. Larger droplets have higher Stokes numbers and therefore respond more slowly to changes in the airflow, exhibiting a greater tendency to maintain their trajectories [25]. SLDs may consequently impinge farther downstream and potentially outside regions protected by conventional Ice Protection Systems (IPS), particularly when splashing occurs and ejected secondary droplets re-enter the airflow and re-impinge farther downstream.

The additional phenomena associated with SLDs may be separated into effects occurring before and during impact. In-flight deformation and breakup (Fig. 2.3a) can be regarded as second-order effects because they indirectly influence impingement by modifying the droplet shape, aerodynamic drag, and trajectory before contact with the surface. By contrast, splashing, bouncing, and secondary-droplet ejection (Fig. 2.3b) constitute first-order wall-interaction effects because they directly determine the fraction of the impinging water mass that remains on the surface [65].

As an SLD travels through a nonuniform flow field, variations in the aerodynamic pressure and shear stresses acting over its surface may deform it from its initially spherical shape. The resistance of the droplet to this deformation is governed primarily by surface-tension and viscous forces. When the aerodynamic forces become sufficiently large relative to these restoring forces, the spherical-droplet assumption is no longer valid, and the droplet may adopt an oblate or more complex nonspherical shape. Under more severe conditions, continued deformation may ultimately lead to breakup into smaller droplets [68].

Droplet deformation can affect impingement by modifying the aerodynamic drag and, consequently, the droplet response time and trajectory. Breakup may additionally alter the



(a) Droplet deformation [66]. (b) Droplet splashing [67].

Figure 2.3 Examples of droplet deformation and splashing phenomena relevant to SLD conditions.

droplet-size distribution and produce smaller fragments that interact more strongly with the airflow. Nevertheless, the influence of deformation and breakup on the resulting impingement distribution has generally been found to remain limited for many icing conditions, with potentially greater importance under freezing-drizzle conditions [65, 69]. For this reason, these mechanisms are often neglected or represented through simplified corrections rather than modeled explicitly.

A commonly used approximation accounts for deformation by modifying the drag coefficient without explicitly resolving the evolving droplet shape. The effective drag coefficient may be obtained by blending correlations for spherical droplets and oblate disks as a function of the droplet Reynolds and Weber numbers [70]:

$$C_D = (1 - f)\, C_{D,\text{sphere}} + f C_{D,\text{disk}} \tag{2.16}$$

with

$$f = 1 - \left(1 + 0.07\sqrt{\text{We}}\right)^{-6} \tag{2.17}$$

where the Weber number is defined as

$$\mathrm{We} = \frac{\rho_a \, D \, |u_a - u_d|^2}{\sigma} \tag{2.18}$$

where σ is the surface tension of water. $C_{D,\mathrm{sphere}}$ and $C_{D,\mathrm{disk}}$ depend on the droplet Reynolds number.

The most consequential SLD effects generally arise during droplet-wall interaction. At relatively low impact energy, an impinging droplet may stick to or spread over the surface and contribute directly to the surface water film. Under other conditions, it may bounce, splash, or break into secondary droplets [71, 72]. These secondary droplets can be ejected from the surface and may subsequently reimpinge farther downstream, thereby extending the effective impingement region. The amount of water deposited on the surface is therefore determined not only by the collection efficiency, but also by the outcome of the impact process.

Because resolving individual droplet impacts with interface-capturing methods would be prohibitively expensive at aircraft scale [73], these first-order SLD wall-interaction effects are generally represented through semi-empirical models. Such models are commonly formulated in terms of nondimensional impact parameters, including the Reynolds, Weber, and Ohnesorge numbers, and may also account for the impact angle and impact velocity. A common approach consists in correcting the collection efficiency using a sticking rate ϵ, or equivalently a mass-loss coefficient f_m:

$$\beta_{\mathrm{final}} = \epsilon \beta_0 = (1 - f_m) \, \beta_0 \tag{2.19}$$

where β_0 is the collection efficiency obtained before accounting for splashing or bouncing. This correction represents the fraction of the impinging water mass that remains on the surface after impact.

More advanced SLD models also account for the generation and reimpingement of secondary droplets. In a Lagrangian framework, this can be handled by reinjecting secondary droplets with prescribed diameters, velocities, and ejection directions obtained from semi-empirical correlations [63]. In an Eulerian framework, secondary reimpingement is more difficult to represent directly because the dispersed phase is described as a continuous field. One strategy consists in modifying the boundary conditions on splashing surfaces so that the secondary droplets are reemitted into the computational domain and transported by the Eulerian equations [74, 75]. The resulting collection efficiency may then include both the retained contribution and the mass associated with secondary reimpingement:

$$\beta_{\text{final}} = \epsilon\beta_0 + \sum \beta_{\text{reimp}} \tag{2.20}$$

where β_{reimp} denotes the collection-efficiency contribution associated with reimpinging secondary droplets. The modeling of SLD effects therefore remains closely linked to the selected droplet formulation. Eulerian methods provide an efficient and robust framework for impingement calculations, but require additional boundary-condition treatments to represent secondary droplets. Lagrangian methods are generally more expensive, but naturally provide the impact quantities required by splashing and bouncing correlations and allow secondary droplets to be tracked explicitly.

Several semi-empirical models have been proposed to describe droplet impact and splashing under SLD conditions [76–78]. Among these, the formulations of Wright et al. [65] and Trontin et al. [69] have been widely adopted in aircraft-icing applications due to their ability to provide reliable predictions against experimental data. Wright et al. [65] incorporated the impact angle into the splashing criterion and calibrated the model using experimental water-collection data. Trontin et al. [69] proposed an alternative formulation in which the mass loss is described through separate dependencies on the impact angle and kinetic energy. The principal characteristics of these two formulations are summarized in Table 2.2, where the splashing criterion defines the condition under which splashing is taken into account.

Table 2.2 SLD splashing models.

Source	**Mass Loss Coefficient**	**Splashing Criterion**
Wright et al. [65]	$0.7(1-\sin\theta_0)\left[1-\exp\left(-0.0092(K_M-200)\right)\right]$	$\dfrac{0.859\,K_M^{1/2}\left(\dfrac{\rho_w}{LWC}\right)^{1/8}}{\sin^{5/4}\theta_0} > 200, \qquad \theta < \dfrac{\pi}{2}$
Trontin et al. [69]	$1-g\left(\dfrac{\theta}{\theta_c}\right) f\left(\dfrac{K_c-K_0}{K_0}\right)$	$K_c = We_n\,Oh^{-2/5} > 657$

2.4 Thermodynamic Modeling

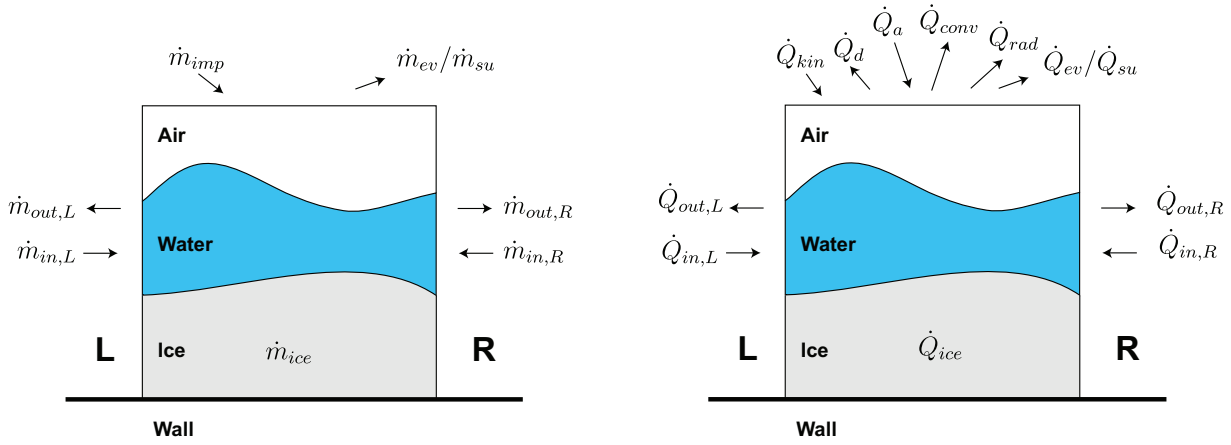
Once the airflow and droplet-impingement solutions have been obtained, a thermodynamic analysis is performed over the exposed surface to determine the local ice-accretion rate. The aerodynamic solution provides the convective heat-transfer coefficient, wall shear stress, and surface-pressure distribution, whereas the dispersed-phase solution provides the local impinging-water mass flux and droplet kinetic energy at impact. These quantities are introduced into surface mass- and energy-balance equations describing the freezing, evaporation, and transport of water over the surface (Fig. 2.4). These balances are written as

$$\dot{m}_{imp} + \sum \dot{m}_{in} = \sum \dot{m}_{out} + \dot{m}_{ev/su} + \dot{m}_{ice} \tag{2.21}$$

and

$$\dot{Q}_{kin} + \dot{Q}_d + \dot{Q}_a + \sum \dot{Q}_{in} = \dot{Q}_{conv} + \dot{Q}_{rad} + \dot{Q}_{ev/su} + \sum \dot{Q}_{out} + \dot{Q}_{ice} \tag{2.22}$$

A detailed description of the individual mass- and energy-balance terms and their corresponding expressions is provided in Appendix A.



(a) Mass balance.

(b) Energy balance.

Figure 2.4 Control volume diagrams for mass and energy balance equations. Adapted from [2].

The thermodynamic problem involves three unknowns: the local ice-accretion rate, $\dot{m}_{\text{ice}}$; the outgoing runback-water flow rate, $\dot{m}_{\text{out}}$; and the surface temperature, T_s. The mass and energy conservation equations alone are insufficient to determine these quantities simultaneously. The system is therefore closed by introducing phase-state conditions derived from the classical Messinger model [79]:

- **Dry regime:** a dry element containing no liquid water;
- **Rime or fully freezing regime:** an element for which all incoming water freezes and $\dot{m}_{\text{out}} = 0$;
- **Glaze or partially freezing regime:** an element covered by a liquid-water film, for which only part of the incoming water freezes and $T_s = T_{\text{freeze}}$;
- **Fully liquid regime:** an element for which no ice is formed and $\dot{m}_{\text{ice}} = 0$.

Although the model remains the basis of many contemporary ice-accretion solvers, its original implementation relied on a surface-marching procedure initiated near the stagnation point. This dependence on a clearly identified stagnation location complicates its application to configurations involving multiple stagnation regions, separated flows, or general 3D geometries.

Several extensions have consequently been proposed to improve either the physical description or the numerical robustness of the thermodynamic calculation. Myers et al. [80], for example, extended the classical formulation by accounting for heat conduction through the growing ice layer. Although this modification provides a more detailed representation of the temperature distribution within the ice, it does not remove the dependence of the solution on the treatment of the stagnation point.

A different strategy was introduced by Zhu et al. [81] through an iterative implementation of the Messinger model. Rather than relying exclusively on a predefined marching direction originating from a stagnation point, the surface-water distribution and thermodynamic state are updated iteratively. The transport direction of the runback water is determined from the local wall-shear-stress direction, with the surface-pressure gradient providing an additional contribution. This formulation is better suited to complex and 3D geometries and has consequently been adopted by several modern ice-prediction tools, as reflected in the methods reported during recent ice-prediction workshops [27, 28].

Bourgault et al. [82] proposed an alternative formulation known as the Shallow-Water Icing Model (SWIM). In this approach, the liquid-water film is represented through surface PDEs describing its transport and evolution. In contrast to the iterative Messinger formulation, SWIM retains an explicitly unsteady description of the water film. It can therefore represent transient film-transport phenomena more naturally, but generally requires a more involved numerical treatment and introduces an additional computational cost. For most ice-accretion conditions, the resulting ice shapes have nevertheless been found to remain broadly comparable to those obtained using iterative Messinger formulations [25]. A more comprehensive assessment and comparison of the classical Messinger model, its extensions, and SWIM formulations is provided by Lavoie et al. [83]. In that work, it was established that, for time scales beyond the transient film response, SWIM recovers the Messinger solution by construction [83].

2.5 Geometry Evolution

Once the local ice-accretion rate has been determined by the thermodynamic solver, the solid surface must be updated to represent the newly accumulated ice. The updated surface then becomes the aerodynamic geometry for the subsequent accretion layer. Geometry-evolution techniques can generally be divided into two broad categories: interface-tracking methods, in which the ice–air interface is represented explicitly, and interface-capturing methods, in which the interface is embedded implicitly within a field defined over a computational domain.

2.5.1 Interface Tracking

Interface-tracking methods represent the ice–air interface using marker points located directly on the surface or within the surrounding control volumes. In ice-accretion simulations, the markers generally correspond to the nodes of the surface mesh. The position of the interface is therefore known explicitly throughout the geometry-update procedure. The most commonly used approach is the algebraic geometry-evolution method, also referred to as Lagrangian node displacement [84]. In its simplest form, each surface node is moved along the local normal by the predicted ice thickness (Fig. 2.5):

$$\boldsymbol{x}_{\text{new}} = \boldsymbol{x}_{\text{old}} + h_{\text{ice}}\boldsymbol{n} \tag{2.23}$$

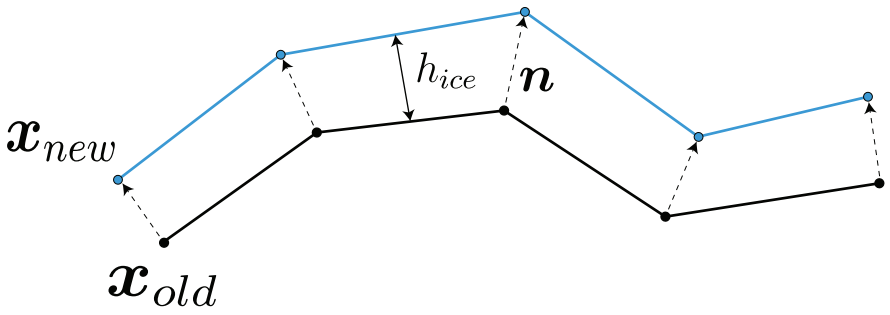


Figure 2.5 Schematic representation of surface-node displacement along the local normal according to the predicted ice thickness.

where $\boldsymbol{x}_{\text{old}}$ and $\boldsymbol{x}_{\text{new}}$ denote the initial and updated node positions, respectively, h_{ice} is the local accumulated ice thickness, and $\boldsymbol{n}$ is the outward surface-normal vector. Several extensions have been introduced to improve the robustness and accuracy of this displacement procedure. One approach consists of dividing the prescribed ice thickness into a sequence of smaller increments and recomputing the surface normals after each intermediate displacement [85].

This results in smoother surface normals and decreases the likelihood of surface-mesh entanglement during geometry evolution. Another formulation that has been proposed consists of deriving the displacement direction from a normal-vector field rather than directly from the discrete surface normals [32, 35]. Such fields are typically obtained from the gradient of a wall-distance field [86]. Since the distance field is computed over the surrounding domain, neighboring displacement directions become naturally aligned and vary smoothly across the surface.

Bourgault-Côté et al. [2] proposed a different interface-tracking approach based on constrained hyperbolic mesh generation. In this method, successive mesh layers are generated outward from the initial surface until the prescribed ice thickness is reached. An advantage of this formulation is that the newly accreted region is directly represented by a volume mesh, allowing the added ice volume to be evaluated without requiring a separate reconstruction procedure. Its application is nevertheless restricted to structured surface meshes, which limits its generality for complex 3D configurations.

The principal advantage of interface-tracking methods is that the interface position is explicitly available, allowing fine geometric structures to be represented accurately when the surface discretization is sufficiently resolved. However, large node displacements may lead to surface-element distortion, self-intersections, and a loss of mesh quality. Topological changes, such as the merging of neighboring ice structures or the formation of highly concave regions, are also difficult to handle robustly using an explicit surface representation. Although several successful treatments have been proposed in the literature [87], maintaining a valid surface and volume mesh remains a challenging aspect of multilayer simulations involving complex ice shapes often requiring additional corrective procedures, such as mesh smoothing, to maintain acceptable mesh quality [6].

2.5.2 Interface Capturing

Interface-capturing methods represent the interface implicitly through a field variable defined over the surrounding computational domain. Among these methods, the level-set approach embeds an n-dimensional interface within an (n+1)-dimensional space and tracked through the evolution of a scalar function ϕ defined in that higher-dimensional domain [88]. The ice–air interface is then recovered as the zero level set of ϕ:

$$\Gamma = \{\boldsymbol{x} \,|\, \phi(\boldsymbol{x}) = 0\}\,. \tag{2.24}$$

As illustrated in Figure 2.6, the level-set function is commonly defined as positive in the fluid

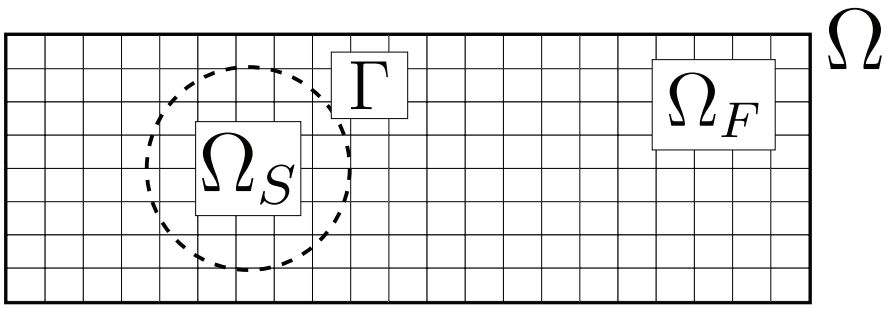


Figure 2.6 Schematic representation of the level-set function. The ice–air interface Γ is implicitly defined by the zero level set ($\phi = 0$), separating the solid domain Ω_S ($\phi < 0$) from the surrounding air domain Ω_F ($\phi > 0$).

region, negative within the solid, and zero at the interface. Its evolution is governed by an advection equation of the form

$$\frac{\partial \phi}{\partial t} + \boldsymbol{V} \cdot \nabla \phi = 0 \tag{2.25}$$

where $\boldsymbol{V}$ is an extension of the interface-growth velocity to the computational region surrounding the interface. When ϕ is maintained as a signed-distance function, the interface-normal vector $\boldsymbol{n}$ and curvature κ can be evaluated as

$$\boldsymbol{n} = \frac{\nabla \phi}{|\nabla \phi|} \tag{2.26}$$

and

$$\kappa = \nabla \cdot \left(\frac{\nabla \phi}{|\nabla \phi|} \right) \tag{2.27}$$

Several attempts have been made to introduce level-set representations into aircraft ice-accretion simulations. Peña et al. [29] presented one of the first applications of a level-set method to the evolution of an ice shape. Their work demonstrated the feasibility of representing ice growth implicitly, but the application was limited to single-step accretion. Beaugendre et al. [89] employed a level-set representation in a different icing context, namely to describe the evolving geometry and trajectories of ice fragments during shedding simulations. In this case, the implicit representation facilitated the treatment of moving solid interfaces and their interaction with the surrounding flow.

Bourgault-Côté et al. [2] and Lavoie et al. [30] subsequently extended the use of level-set methods to 2D multilayer ice-accretion simulations. Their studies considered both body-fitted and immersed-boundary frameworks.

An alternative formulation was proposed by Donizetti et al. [90], who introduced a discrete level-set representation constructed from a Lagrangian description of the surface. In this approach, the interface remains explicitly represented, while the level-set values at surrounding mesh points are recovered geometrically from the distance to the Lagrangian description of the surface. The level-set field is therefore used primarily as an implicit geometric description rather than as the solution of a continuous advection equation.

Level-set methods offer several advantages for ice-accretion simulations. They can accommodate large interface displacements and topological changes without requiring the connectivity of an explicit surface mesh to remain unchanged. These properties make them attractive for the development of robust 3D multilayer geometry-evolution procedures. However, the signed-distance property

$$|\nabla\phi| = 1 \tag{2.28}$$

is generally not preserved as the interface evolves, particularly when the interface undergoes large displacements. A reinitialization or redistancing procedure may therefore be required to recover a suitable distance field [91]. The frequency at which this procedure should be applied depends on the numerical formulation and the evolution of the interface. Furthermore, reinitialization must be performed carefully, since it may displace the interface, alter the enclosed ice volume, or smooth sharp geometric features [42]. The updated surface must then be extracted from the implicit level-set representation before the surrounding volume mesh can be regenerated. Particular care is required during this surface-reconstruction and volume-remeshing process to preserve distinctive features of the original clean geometry, especially in regions unaffected by ice accretion, while accurately representing the newly formed ice structures. In addition, numerical difficulties associated with explicit time integration of the level-set equation have been reported, with stability constraints potentially imposing restrictive time-step requirements during interface evolution [2].

2.6 Geometry Representation

The robustness of aircraft icing simulations is strongly influenced by the geometry-evolution and mesh-update procedures. In conventional body-fitted approaches, the computational mesh must conform to the aircraft surface and must therefore be deformed or regenerated after each accretion layer. As the ice shape develops, the surface may exhibit regions of high curvature, horns, scallops, and other localized irregularities that make it increasingly difficult to preserve high-quality near-wall elements and avoid mesh degeneration or failure.

Immersed-boundary methods provide an alternative framework in which the solid geometry is represented independently of the underlying computational mesh. The origins of the immersed-boundary approach are commonly traced to the pioneering work of Charles Peskin [92], who introduced forcing terms into the Navier–Stokes equations to represent the interaction between blood flow and the moving boundaries of heart valves. Since this early contribution, a wide range of immersed-boundary formulations has been developed.

In an immersed-boundary framework, the physical boundary is generally allowed to intersect the computational cells rather than coincide with mesh faces. The boundary conditions may then be enforced through modifications of the governing equations, the numerical fluxes, or the discrete solution near the interface. Because the mesh does not need to conform explicitly to the surface, these methods can substantially simplify grid generation and geometry updates.

Immersed-boundary methods have gained considerable attention and have been applied to a broad range of problems, including inviscid flows [93, 94], turbulent flows [95, 96], multiphase flows [97, 98], and fluid–structure interaction [99–101]. More recently, they have also been introduced into aircraft ice-accretion simulations [30, 32, 102, 102].

Their use in aircraft icing, however, remains comparatively limited. In particular, immersed-boundary approaches were reported by only one participant in each of the first two Ice Prediction Workshops, indicating that their application to complete ice-accretion frameworks is still at an early stage of development [27, 28]. Additional work is therefore required to assess their accuracy, robustness, and compatibility within a coupled multilayer ice-accretion framework.

Following the classification proposed by Sotiropoulos and Yang [103], immersed-boundary methods may be divided into three broad families: geometrical methods, continuous-forcing methods, and discrete-forcing methods. Geometrical methods modify the computational mesh, control volumes, and numerical integration procedure in the vicinity of the immersed interface. Continuous-forcing methods introduce forcing or source terms into the governing equations to represent the effect of the solid boundary. Discrete-forcing methods instead impose the boundary conditions directly within the discretized equations, typically by reconstructing or modifying the solution near the interface. The following sections summarize the main characteristics of these three approaches.

2.6.1 Geometrical Methods

The principal geometrical approach is the cut-cell method. In this formulation, the solid interface is represented by geometrically modifying the cells of a background mesh that it intersects. The portions of these cells located inside the solid are removed, producing control volumes that conform locally to the immersed boundary while leaving the remainder of the mesh unchanged. One of the earliest cut-cell formulations was introduced by Clarke et al. [3] for Euler-flow simulations around multi-element airfoils. A general illustration of the cut-cell approach is provided in Figure2.7.

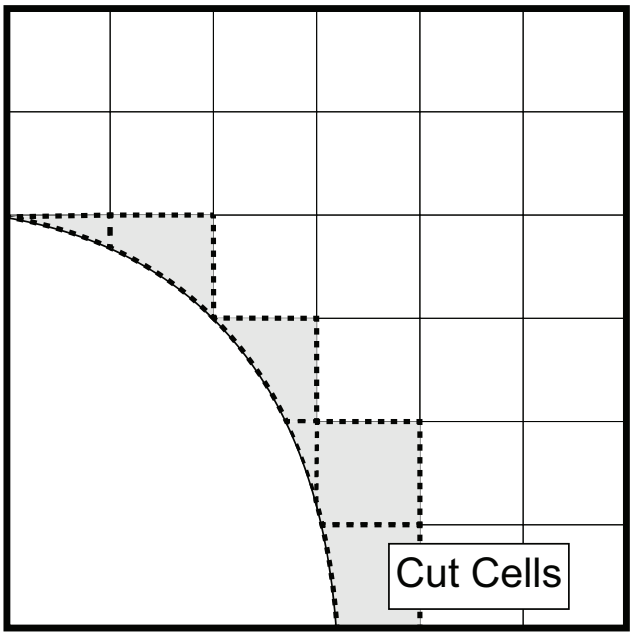


Figure 2.7 Illustration of the boundary cut-cell method [3].

Because the interface is represented through an actual modification of the computational cells, some authors treat cut-cell methods separately and do not classify them strictly as immersed-boundary methods.

Cut-cell methods provide a sharp representation of the interface and allow boundary conditions and numerical fluxes to be imposed directly on the reconstructed faces. They are also naturally suited to conservative finite-volume formulations, although the solver must support general polygonal or polyhedral control volumes.

Their main limitations are the complexity of the geometric preprocessing, particularly in 3D, and the formation of very small cut cells when the interface passes close to a mesh node or face. These cells may impose severe time-step restrictions in explicit schemes and degrade numerical stability. To alleviate these problems, several remedies have been proposed in the

literature, including cell merging [104], cell linking [105], and conservative cell mixing [106].

2.6.2 Continuous-Forcing Methods

Continuous-forcing methods impose the immersed-boundary conditions by introducing a distributed force directly into the continuous governing equations. For example, the incompressible Navier–Stokes equations may be written as

$$\nabla \cdot \boldsymbol{u} = 0 \tag{2.29a}$$

$$\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \boldsymbol{u} + \boldsymbol{f} \tag{2.29b}$$

where $\boldsymbol{u}$ is the fluid velocity, ρ is the density, p is the pressure, ν is the kinematic viscosity, and $\boldsymbol{f}$ is the forcing term used to represent the immersed boundary. The expression of $\boldsymbol{f}$ distinguishes the different continuous-forcing formulations.

The original immersed-boundary method was introduced by Peskin for the simulation of blood flow interacting with elastic heart structures [92]. The fluid equations are solved on an Eulerian background mesh, whereas the immersed boundary is represented by massless Lagrangian markers with coordinates $\boldsymbol{X}(s,t)$, where s denotes the curvilinear coordinate along the immersed boundary and t is time. The force exerted by the boundary on the fluid is distributed to the surrounding Eulerian points through a regularized Dirac function:

$$\boldsymbol{f}(\boldsymbol{x},t) = \int_\Gamma \boldsymbol{F}(s,t)\delta(\boldsymbol{x} - \boldsymbol{X}(s,t))\ \mathrm{d}s \tag{2.30}$$

where $\boldsymbol{F}$ is the force defined on the Lagrangian boundary Γ. Conversely, the velocity of the markers is interpolated from the Eulerian velocity field according to

$$\frac{\partial \boldsymbol{X}(s,t)}{\partial t} = \int_\Omega \boldsymbol{u}(\boldsymbol{x},t)\delta(\boldsymbol{x} - \boldsymbol{X}(s,t))\ \mathrm{d}\boldsymbol{x} \tag{2.31}$$

This two-way transfer allows the fluid to displace the immersed boundary while the boundary exerts a force on the fluid. The regularized Dirac function spreads this interaction over several grid points, resulting in a diffuse rather than sharp representation of the interface. Although this formulation is well suited to deformable interfaces, its extension to rigid bodies is more problematic. As the boundary stiffness is increased to suppress deformation, the constitutive laws used to determine the restoring force become increasingly stiff and may become ill posed

in the rigid-body limit [107].

These limitations motivated the development of penalization methods for rigid boundaries. Initially introduced by Arquis and Caltagirone to model flow through porous media [108], the method was subsequently extended to represent solid obstacles as regions of very low permeability [109]. A mask function (Fig. 2.8) distinguishes the solid and fluid domains:

$$\chi(\boldsymbol{x}, t) = \begin{cases} 1, & \boldsymbol{x} \in \Omega_s \\ 0, & \boldsymbol{x} \in \Omega_f \end{cases} \tag{2.32}$$

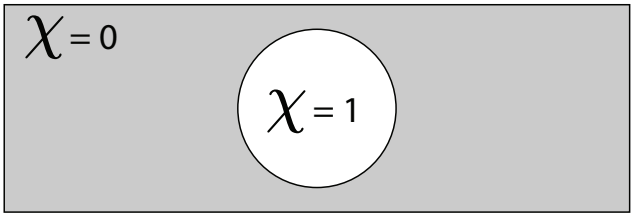


Figure 2.8 Schematic representation of the solid and fluid domains using the mask function χ.

For the incompressible Navier–Stokes equations, penalization is commonly applied to the momentum equation to enforce the no-slip boundary condition. This is achieved by introducing the forcing term

$$\boldsymbol{f} = \frac{\chi}{\eta} (\boldsymbol{u}_s - \boldsymbol{u}) \tag{2.33}$$

where $\boldsymbol{u}_s$ is the prescribed solid velocity and η is a permeability or penalization parameter. In the fluid region, $\chi = 0$ and the original equations are recovered. Within the solid, $\chi = 1$, and a sufficiently small value of η ($\approx 10^{-8}$ [110]) drives the fluid velocity toward the prescribed solid velocity.

Penalization methods require only limited modifications to an existing solver, with the main preprocessing operation consisting of identifying the solid region and constructing the mask function. They also possess a well-established mathematical foundation and can be combined with a wide range of spatial discretizations [111]. However, the boundary is generally represented diffusely, which commonly limits the classical formulation to first-order spatial accuracy near the interface. Higher-order corrections have nevertheless been proposed [112]. In the context of aircraft icing, Lavoie et al. [30] successfully extended the penalization approach to the Eulerian droplet equations, demonstrating its applicability to droplet-impingement

prediction on immersed geometries.

Finally, the penalization parameter must be selected carefully: Decreasing η improves the enforcement of the boundary condition but increases the stiffness of the governing equations and may therefore require careful numerical treatment. Although the example presented above involves a single penalization term, the enforcement of other boundary conditions may require several distinct penalization parameters. Their simultaneous selection can introduce additional case-dependent tuning and may require manual adjustment [113].

2.6.3 Discrete-Forcing Methods

Discrete-forcing methods impose the immersed-boundary conditions directly within the discretized governing equations. Their development is commonly traced to the work of Mohd-Yusof [114], who modified the discrete momentum equations near the immersed interface to enforce a prescribed velocity. This approach was subsequently extended to other numerical frameworks, including finite-difference and finite-volume methods [115,116]. For illustration, the incompressible momentum equation may be written as

$$\frac{\partial \boldsymbol{u}}{\partial t} = \boldsymbol{R} + \boldsymbol{f} \tag{2.34}$$

where $\boldsymbol{R}$ contains the convective, pressure, and viscous contributions. and $\boldsymbol{f}$ denotes the discrete forcing. Using a forward-Euler time discretization gives

$$\frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^{n}}{\Delta t} = \boldsymbol{R} + \boldsymbol{f} \tag{2.35}$$

Imposing a target velocity $\boldsymbol{u}_{\text{target}}$ at the immerse-boundary Γ leads to

$$\boldsymbol{f} = \begin{cases} -\boldsymbol{R}^{n} + \dfrac{1}{\Delta t}\left(\boldsymbol{u}_{\text{target}} - \boldsymbol{u}^{n}\right) & \text{on } \Gamma \\ 0 & \text{elsewhere} \end{cases} \tag{2.36}$$

Although the formulation is expressed through a forcing term, this term does not need to be explicitly computed. Instead, the velocity at the cell is directly updated to enforce the desired target value. Majumdar et al. [117] interpreted this approach as a generalization of the ghost-cell method traditionally used in body-fitted simulations to complete the numerical stencil near solid boundaries. The term Ghost-Cell Immersed Boundary Method (GCIBM) was later introduced by Tseng and Ferziger [4].

Figure 2.9 illustrates the general principle of the ghost-cell approach near an immersed bound-

ary. The fluid cells adjacent to the immersed interface require values from cells located inside the solid to complete their computational stencil, similarly to the use of halo cells at body-fitted boundaries. These cells are therefore identified as ghost cells, and appropriate values must be assigned to them to account for the presence of the immersed boundary. The manner in which these values are determined directly influences the accuracy and stability of the resulting solution.

The simplest approach consists of directly assigning the prescribed boundary value as the target value in the ghost cells. However, this produces a staircase representation of the immersed interface, limits the local accuracy to first order, and can induce spurious oscillations near the boundary. A smoother treatment can be obtained by weighting the forcing according to the fraction of each computational cell occupied by the solid. A more widely adopted approach instead determines the ghost-cell values through interpolation from neighboring fluid cells while accounting for the boundary condition at the immersed interface [4, 115].

One common interpolation procedure relies on the construction of an image point in the fluid domain. For each ghost cell, a wall point is first determined on the immersed interface and an image point is constructed in the fluid, commonly along the wall-normal direction, as illustrated in Fig. 2.9. Since the image point generally does not coincide with a computational point, its value is reconstructed from neighboring fluid cells. The ghost-cell value can then be determined using the reconstructed image-point value together with the prescribed boundary condition at the wall.

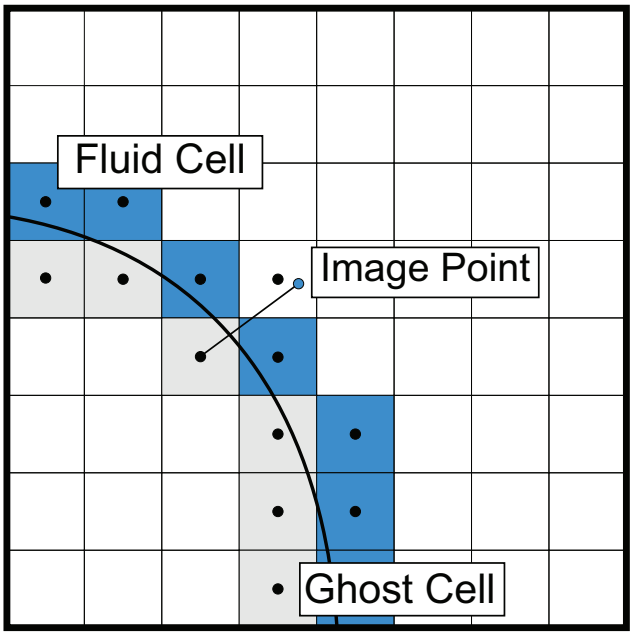


Figure 2.9 Image-point and ghost-cell construction near an immersed boundary [4].

Depending on the interpolation procedure, ghost-cell methods can preserve the order of accuracy of the underlying numerical scheme and provide a sharp representation of the immersed interface [4, 118]. The governing equations and flux calculations in the fluid domain remain largely unchanged, with the principal additions consisting of the geometrical preprocessing and the interpolation used to update the image and ghost values. These operations must be performed efficiently to limit the computational overhead.

The main difficulties arise from the construction of suitable ghost, wall, and image points in regions containing high curvature, sharp features, or closely spaced boundaries. The image point may fall inside the solid or may not have a sufficient number of neighboring fluid cells for the required interpolation stencil. The number and placement of ghost-cell layers also depend on the numerical scheme and the width of its computational stencil. These issues become particularly important for complex iced geometries.

In addition to ghost-cell methods, other discrete-forcing formulations have been developed. Capizzano et al. [119, 120] proposed a face-forcing approach in which the boundary condition is imposed through the fluxes of the cell faces intersected by the immersed interface rather than through reconstructed cell values. The procedure remains similar to the ghost-cell approach: a wall-normal direction is constructed through the intersected face, an image point is defined in the fluid, and the state at this point is interpolated from neighboring cells. The reconstructed state is then used to determine the flux at the immersed-boundary face. This approach has been applied to Euler flows and RANS simulations [119, 120].

Face forcing avoids some of the difficulties associated with placing several rows of ghost cells near complex interfaces. However, it requires a dedicated treatment of the numerical fluxes on immersed-boundary faces and therefore introduces more extensive modifications to the flow solver than conventional ghost-cell methods.

2.7 Synthesis and Positioning of the Present Work

Despite the continued improvement of the individual modules involved in numerical ice-accretion prediction, the automation and robustness of complete multilayer frameworks remain limited by three closely connected aspects: droplet-impingement modeling, geometry evolution, and the representation of the evolving geometry within the computational mesh. These components are particularly important in 3D simulations, where accurately representing the droplet trajectories and their interaction with increasingly complex iced geometries becomes more challenging, while maintaining a valid surface and a high-quality volume mesh becomes progressively more difficult as the ice shape develops. At the same time, the large

meshes required for 3D configurations and the repeated solution of the coupled physical models over multiple accretion steps impose substantial computational requirements. Consequently, the numerical methods employed within each module must not only remain robust as the geometry evolves, but also be compatible with efficient parallel execution on modern high-performance computing architectures.

Regarding droplet impingement, the literature does not identify a universally superior formulation. Eulerian and Lagrangian approaches provide comparable access to the collection-efficiency distribution, but each presents distinct advantages and limitations. Eulerian methods integrate naturally within finite-volume CFD solvers, are readily parallelized, and remain attractive for large 3D calculations. Their continuum description, however, introduces numerical diffusion near impingement limits and does not directly provide individual impact quantities. Lagrangian methods offer a more direct physical representation of droplet motion and naturally provide impact locations, velocities, and angles. These properties are particularly valuable for SLD modeling and for tracking droplets generated by splashing. Their main limitation is computational cost, especially since a large number of trajectories must be tracked to obtain sufficiently resolved impingement maps. For large-scale 3D applications, efficient particle localization, trajectory integration, and parallel workload distribution therefore become important considerations for maintaining computational performance. Consequently, the choice between Eulerian and Lagrangian formulations remains dependent on the intended application rather than on a clear overall advantage of either approach.

The literature on geometry evolution reveals a growing interest in interface-capturing methods. Classical interface-tracking approaches remain widely used because of their simplicity and because the evolving surface is available explicitly. Nevertheless, their robustness deteriorates when large displacements, concave regions, sharp features, or surface self-intersections develop. These difficulties become increasingly restrictive in 3D multilayer simulations, where maintaining a valid surface connectivity is considerably more involved than in 2D. Interface-capturing techniques, particularly level-set methods, provide a more flexible alternative because they represent the interface implicitly and can accommodate large deformations and topological changes without explicitly preserving the original surface connectivity. They are therefore well suited to the formation of complex ice morphologies. Their benefits are accompanied by challenges related to signed-distance preservation, preservation of sharp features, and the extraction of an explicit surface required for the subsequent accretion layer.

Geometry representation and volume-mesh generation constitute the third central difficulty. In conventional body-fitted frameworks, the volume mesh must be regenerated or deformed after each accretion step. Although this treatment provides a direct and accurate application

of the wall boundary conditions, its automation becomes increasingly difficult as the ice shape develops sharp horns, scallops, and highly concave regions. Immersed-boundary methods have therefore gained considerable attention because they decouple the evolving surface from the underlying volume mesh and can reduce, or in some cases eliminate, repeated body-fitted remeshing.

No single immersed-boundary formulation is equally suited to all the equations involved in an ice-accretion framework. Penalization methods require relatively limited modifications to an existing solver and have proved particularly attractive for Eulerian droplet transport [32,110]. Their extension to aerodynamic simulations involving high-Reynolds-number turbulent flows is less straightforward.

Discrete approaches, such as ghost-cell immersed-boundary methods, provide a sharper representation of the interface and can retain the formal accuracy of the underlying numerical scheme. They are therefore generally better suited to aerodynamic-flow calculations, including compressible and turbulent formulations [32]. Their implementation, however, requires more elaborate geometrical preprocessing and robust interpolation procedures. In particular, the construction of ghost, wall, and image points becomes challenging near sharp ice features, highly curved surfaces, and closely spaced portions of the interface.

The preceding review therefore suggests that the most appropriate methodology for a complete ice-accretion framework may involve different numerical treatments for its individual modules rather than the systematic use of a single formulation. Moreover, these treatments must remain computationally tractable and scalable as simulations progress toward increasingly complex 3D configurations on modern HPC systems. The present thesis follows this direction by focusing on complementary developments for droplet impingement, geometry evolution, and geometry representation. First, a Lagrangian droplet-tracking framework is investigated as an alternative to the established Eulerian formulation, with particular attention to its computational efficiency, parallel scalability, collection-efficiency prediction, and ability to represent SLD wall interactions. Second, a level-set method is developed for robust 3D multilayer geometry evolution, with the objective of reducing the dependence of the update procedure on the connectivity of an explicit surface mesh. Finally, the integration of the evolving geometry within an immersed-boundary framework is examined to assess the extent to which repeated body-fitted remeshing can be reduced.